



MG-CRAG: fusion of multi-granular retrieval evaluators in corrective RAG with weakly supervised fine-tuning

Negin Masoumi¹ · Omid Davar¹ · Mahdi Eftekhari¹

Received: 5 July 2025 / Revised: 4 November 2025 / Accepted: 13 April 2026

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2026

Abstract

This paper introduces multi-granular corrective retrieval-augmented generation (MG-CRAG), a novel framework that enhances response quality in retrieval-based systems by processing text at multiple levels of granularity. Building on recent CRAG approaches that mitigate hallucinations in large language models through irrelevant content filtering, our method addresses the limitations of heuristic labeling via a weakly supervised, four-stage pipeline that combines manual annotation with autoencoder-guided pseudo-labeling. The framework employs a sequential passage-level retrieval evaluator and sentence-level retrieval evaluator, both based on efficient T5 architectures, to hierarchically refine documents. The short-answer datasets used in this study include ARC-Challenge, PubHealth, and PopQA. MG-CRAG achieves state-of-the-art performance on ARC-Challenge (68.85% accuracy) and PopQA (59.89% accuracy), while delivering equal results on the PubHealth dataset despite a lower web search rate. Key advantages include significantly reduced dependence on web search, minimal labeled data requirements, and customizable inference modes (strict/moderate/lenient) that optimize performance across different dataset characteristics. The framework also enables tunable trade-offs between accuracy and web search usage, demonstrating that multi-granular processing enhances focus on relevant content, substantially improving answer accuracy while maintaining computational efficiency.

Keywords Large language models · Retrieval-augmented generation · Corrective RAG · Weakly supervised learning · Text clustering · Hallucination mitigation

1 Introduction

LLMs have significantly advanced Natural Language Processing (NLP) by enhancing applications such as text generation, translation, and summarization through improved precision and productivity. However, a considerable challenge associated with LLMs is their propensity to generate content that is factually incorrect or nonsensical, a phenomenon referred to as hallucination. For instance, an LLM might fabricate a historical event that never occurred.

The code is available at <https://github.com/omidacoder/mg-crag>.

✉ Mahdi Eftekhari
m.eftekhari@uk.ac.ir

¹ Department of Computer Engineering, Shahid Bahonar University of Kerman, Kerman, Iran

These hallucinations can manifest as fabricated facts, logical inconsistencies, or contextually irrelevant statements. Mitigation strategies include refining training methodologies to improve data quality, integrating external knowledge bases to ensure factual accuracy, and developing techniques such as real-time fact-checking algorithms to detect and correct inaccuracies in generated content. Despite these efforts, the elimination of hallucinations remains a significant challenge in the field of artificial intelligence due to the complexity of language and the vast amount of data processed by LLMs [1]. This issue raises concerns about the reliability of LLMs in real-world applications, particularly in critical decision-making processes where accuracy is paramount [2].

To tackle the challenge of hallucinations in LLMs, researchers have developed techniques such as RAG [3]. RAG enhances LLMs by integrating external data sources, thereby ensuring that the generated content is factual and reducing inaccuracies. Naive RAG retrieves documents based on a user's query and utilizes this information to generate responses. In contrast, advanced RAG employs sophisticated methods to improve both retrieval and generation processes. These methods include intelligent query intent understanding, modular data retrieval skills, contextual memory for coherent interactions, and augmented generation for accurate and context-aware responses [4].

Building upon the foundations of RAG, the Self-Reflective RAG (Self-RAG) framework introduces an innovative approach to improve LLMs by integrating retrieval, generation, and self-reflection processes, which facilitates more precise and contextually relevant information processing. Self-RAG addresses the limitations of traditional RAG methods, which often retrieve a fixed number of passages without assessing their relevance, potentially leading to inaccurate responses [5]. Self-RAG advances LLM development by integrating self-reflection into retrieval and generation. This approach improves accuracy in handling complex tasks—such as multifaceted questions or context-dependent retrieval by better addressing nuanced information needs.

Traditional RAG systems, which are designed to augment the capabilities of language models by incorporating external information, depend heavily on the relevance of the retrieved documents. This dependency can result in sub-optimal or incorrect responses if the retrieval process fails. To mitigate this issue, CRAG is an advanced framework intended to enhance the robustness and accuracy of LLMs by addressing potential shortcomings in the retrieval process [6]. CRAG introduces a retrieval evaluator that assesses the overall quality of retrieved documents for a given query by analyzing factors such as relevance, accuracy, and completeness of the information. This evaluator assigns a confidence score to the retrieved information, enabling the system to determine the necessity of additional retrieval actions. If the initial retrieval results are insufficient or low-quality, CRAG can activate alternative strategies like web searches to gather more information. These approaches expand both the scope and sources of available data compared to the initial retrieval. Furthermore, CRAG employs a decompose-then-recompose algorithm, which involves breaking down the retrieved documents into smaller, manageable parts to identify key information, and then recomposing these parts to form a coherent and relevant response. This methodology enables the model to selectively focus on key information while filtering out irrelevant content, thereby enhancing the accuracy of the generated responses. Experiments on multiple datasets, encompassing both short- and long-form generation tasks, have demonstrated that CRAG significantly improves the efficacy of RAG-based systems, leading to more reliable and accurate outputs. This paper extends the CRAG framework by investigating several unresolved questions in its methodology. While CRAG represents significant progress, we identify key areas requiring further examination to optimize its performance, as detailed below:

Data Quality for Fine-Tuning the Retrieval Evaluator: The dataset employed for fine-tuning the retrieval evaluator in CRAG relies on heuristic labels instead of human-generated annotations. An examination of the dataset reveals a substantial number of instances containing incorrect labels. Fine-tuning a model on such noisy data may negatively impact its performance. This observation raises the question of whether a small set of high-quality, human-labeled data could be utilized to generate pseudo-labels for a larger dataset, thereby enabling the training of a retrieval evaluator through a weakly supervised approach.

Threshold Determination for the Retrieval Evaluator: The retrieval evaluator trained in CRAG uses a final layer producing scalar values between -1 and 1. For real-world deployment, this requires setting upper/lower thresholds to handle ambiguity - an approach that may prove impractical and limiting. This limitation stems from the binary (0/1) training labels, which prevent direct three-class classification. However, using weakly supervised training enables three-class classification, eliminating the need for explicit threshold definitions across different tasks.

Granularity in Knowledge Refinement: In CRAG, documents are segmented into smaller units termed “strip” for evaluation, a process referred to as knowledge refinement. The issue of granularity in RAG operations has been discussed in [7]. In this paper, two retrieval evaluators are utilized to refine documents at both the passage and sentence levels before they are fed to an LLM, thereby enhancing the granularity of the refinement process.

Web Search Dependency and Retrieval Efficiency: Web search is considered a heavy operation, and if a framework can reduce its dependency on the web search, it will achieve better speed and performance. CRAG initiates web searches when the retrieved documents are irrelevant or incorrect. The frequency of these web searches is influenced by the number of documents classified as correct, which can be controlled by adjusting the upper threshold parameter. A retrieval evaluator that applies stricter criteria for correctness tends to increase the rate of web searches, whereas a more moderate evaluator reduces this rate. In the present approach, a sequential combination of two retrieval evaluators is employed, resulting in a lower web search rate compared to passage-level processing alone. Shorter text segments are generally more likely to be classified as correct, thereby further decreasing reliance on web searches.

System Performance Improvement: By leveraging smaller models and optimized techniques, system performance has been significantly improved while preserving robustness. Models with fewer parameters have been utilized to enhance response speed while maintaining acceptable performance levels. The Self-RAG framework employs an 800M-parameter model for evaluations. Initially, all documents are assessed and classified into one of three categories: Correct, Incorrect, or Ambiguous. All documents are then segmented into strips, which are re-evaluated to identify the top-n for the model’s use. In the proposed method, a more efficient approach has been adopted by utilizing the T5-GTR model, with 300M parameters, alongside a Residual Network containing 17M parameters. Additionally, a pre-trained re-ranker model with 100M parameters has been incorporated. This configuration substantially reduces both training and inference computations compared to the baseline. In light of these considerations, the proposed methods in this paper serve as complementary enhancements to the CRAG framework.

2 Related works

Our work improves the CRAG framework, an advanced RAG method that reduces hallucinations while retrieving relevant information from external documents. This section covers: (1) hallucinations in LLMs, (2) deep text clustering methods, and (3) RAG advancements. Hallucinations in LLMs, where models produce factually inaccurate or misleading outputs, undermine their reliability in critical applications. Foundational research [1, 8] attributes hallucinations to model architecture limitations, such as the softmax bottleneck, and training data issues, including biases and overfitting. Comprehensive reviews [9] classify hallucination sources and propose mitigation strategies, highlighting risks in high-stakes domains like medical diagnosis and legal analysis [10, 11]. Effective mitigation techniques include fine-tuning on preference datasets, reducing hallucinations by up to 96% in translation tasks [12], alongside methods like Reinforcement Learning from Human Feedback (RLHF) [13], adversarial training [14], and adaptive prompt tuning [15]. These approaches improve the accuracy and alignment of facts with human expectations, improving the trustworthiness of LLM.

Text Clustering is essential for organizing unstructured data in NLP applications. Traditional algorithms like K -Means have been enhanced to tackle high-dimensional text challenges. For instance, sparse K -Means offers theoretical guarantees for consistency [16], while deep neural networks combined with K -Means produce coherent clusters [17]. Autoencoders enable dimensionality reduction while preserving semantic relationships. Masked autoencoders improve graph clustering [18], and contrastive learning enhances discriminative representations [19]. The Dual Adversarial Auto-Encoder prevents mode collapse, achieving near-supervised accuracy [20]. Self-supervised learning further leverages clustering properties, with methods like Representation Soft Assignment (ReSA) utilizing encoder outputs to enhance semantically rich and stable clusters in a self-guided manner [21]. Transformer-based models and LLMs introduce novel clustering paradigms. Dynamic clustering in LLM's hidden spaces improves generalization [22], and transformer embeddings enable semantic topic modeling [23]. Efficient dynamic clustering in RAG compresses documents by leveraging latent inter-document relationships, reducing noise and redundancy [24]. The K-LLMmeans algorithm [25] enhances K -Means with LLM-generated centroids for scalability. Interpretability is improved through attention mechanisms [26] and spectral clustering frameworks [27]. Multi-modal clustering aligns cross-modal representations, achieving state-of-the-art performance in tasks like text-to-video retrieval [28].

RAG mitigates hallucinations by incorporating external knowledge into LLMs, grounding outputs in verifiable facts. Pioneering work by [3] introduced retrieval mechanisms to reduce erroneous outputs. Recent advancements include multi-layered retrieval frameworks for context-adaptive multi-hop reasoning [4]. Innovations like RankRAG [29] unify context ranking and text generation to enhance hallucination reduction. Self-RAG [5] employs real-time self-assessment, while CRAG [6] uses lightweight evaluators to trigger corrective actions. Query-Refined RAG (QR-RAG) [30] optimizes retrieval through reinforcement learning-based query refinement. To improve scalability, proposition-based retrieval eliminates extraneous details [7], and MiniRAG [31] achieves 92% of state-of-the-art hallucination reduction with 15x fewer parameters and 40% lower latency. GraphRAG [32] leverages graph-structured data with Graph Neural Networks for nuanced retrieval in domains like drug discovery. Security-focused frameworks, such as SafeRAG [33], address vulnerabilities like data poisoning, while Agentic RAG [34] and [35] enhance precision and robustness

through autonomous agents and provenance-aware retrieval, reducing factual inconsistencies by up to 41%.

In addition to the previously mentioned advances, recent studies have proposed ensemble-learning strategies integrated with RAG frameworks, specifically designed to reduce redundancy in automatically generated examination questions. These methods enhance topical diversity by initially applying fuzzy ontology-mapping techniques to derive enriched entity relationship sets from external knowledge graphs, subsequently using these sets in a RAG model for source text expansion, and finally employing a pretrained automatic question generation (AQG) model (e.g., T5) to generate questions with fewer duplicates [36]. Furthermore, ontology-mapping frameworks tailored for RAG-based modeling have been advanced to mitigate factual hallucinations in pretrained language-model-based question generation. These frameworks are particularly focused on multi-hop question generation scenarios, where spurious named entities or false positive phrases frequently appear, and leverage ontology mapping to enhance factual accuracy and coherence of the generated questions [37].

More broadly, a recent wave of research has introduced novel frameworks aimed at further reducing hallucinations and strengthening factual grounding in RAG systems. For example, THaMES provides an end-to-end toolkit that unifies hallucination detection, automated test set creation, benchmarking, and mitigation strategy selection (e.g., ICL, RAG, PEFT) into a single pipeline, thereby enabling systematic evaluation and deployment of hallucination control methods across models and domains [38]. Concurrently, Ontology-Grounded RAG (OG-RAG) constructs domain-specific ontologies, encodes documents as hypergraphs aligned with these ontologies, and then retrieves a minimal set of ontology-grounded facts to feed into the language model; this approach reports significantly improved fact recall and correctness compared to conventional RAG baselines [39].

Additional contributions include domain-specific applications such as ontology-based RAG for additive manufacturing, which demonstrates how domain ontologies can ground generative AI models in technical workflows, improving interpretability and adherence to process constraints [40]. Another line of inquiry, DRAGON-AI, focuses on dynamic ontology generation using RAG to extract both structural and textual ontology components from unstructured sources, thus facilitating the update or extension of domain knowledge bases without full manual curation [41]. From the ontology-learning perspective, the LLMs4OL challenge investigates how large language models themselves can support ontology learning (i.e., extracting concepts and relations), which in turn feeds improved retrieval grounding in downstream tasks [42].

Moreover, KROMA introduces a hybrid method for ontology matching by embedding retrieval into a language model-based matching pipeline: The system uses targeted knowledge retrieval and prompts enrichment to refine large-scale concept alignment [43]. More recently, HalluGuard proposes a lightweight reasoning model that acts as a filter within the RAG pipeline: It classifies document–claim pairs as grounded or hallucinated and then produces evidence-based justifications—thereby serving as a checkpoint for factual consistency in generated outputs [44]. In the realm of graph and knowledge representations, Distill-SynthKG develops a workflow that distills large-scale knowledge-graph synthesis from extensive corpora and subsequently employs those graphs for downstream graph-based retrieval in RAG systems, improving coverage and retrieval efficiency [45].

Together, these works point to promising future directions: namely, the coupling of retrieval with structured knowledge (such as ontologies and knowledge graphs), the integration of lightweight reasoning or verification modules, and the dynamic construction or refinement of knowledge structures to enable more reliable, less hallucination-prone generative models.

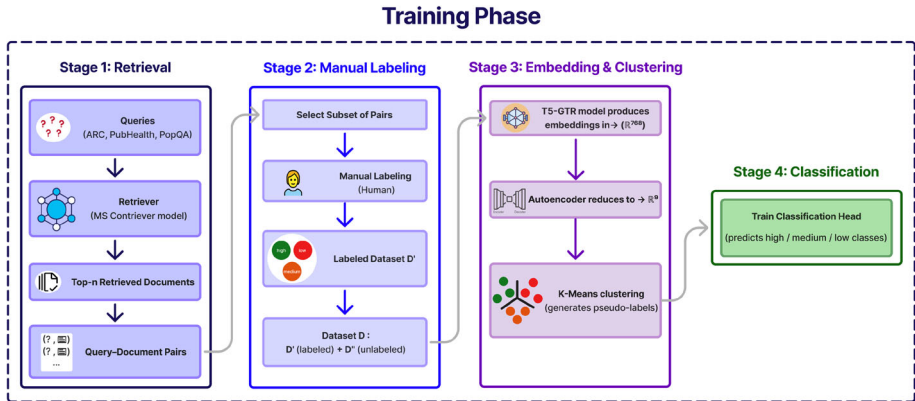


Fig. 1 Training pipeline for retrieval evaluator in MG-CRAG that combines retrieval (MS Contriever) and embedding refinement (autoencoder, quality-based indexing) with corrective filtering. Weakly supervised training is used for preparing the labeled data and then supervised training of classification head

3 MG-CRAG methodology

The CRAG framework [6] was developed to improve response quality in retrieval-based systems by integrating a post-retrieval processing step to filter out useless parts of documents and add useful information from web search. The authors of the CRAG paper used PopQA golden subjects to train the retrieval evaluator module, which includes labels of 0 and 1 based on the presence of subjects in documents using the exact matching technique. However, these heuristic labels were generated without manual verification and have been found to contain errors when they have been reviewed by us. In the proposed method, we tried to minimize these errors by using a weakly supervised method. The work is broadly divided into two main phases: (1) Training Phase and (2) Inference Phase. As shown in Fig. 1, the training phase of the proposed method consists of four stages, each of which influences the final experimental results. In the first stage, (*query*, *document*) pairs are prepared using the retriever model (MS-Contriever). Queries are fed into the retriever model, which retrieves the top-*n* most relevant documents. These pairs of queries and retrieved documents are then compiled for further processing. In the second stage, a small subset of these (*query*, *document*) pairs is manually labeled by an annotator. The labels—*Low*, *Medium*, and *High*—indicate the likelihood of the correct answer to the query, being in retrieved documents. To evaluate document quality in our framework, documents are categorized as follows:

- **Low labeled:** These documents fail to adequately address the query, often including irrelevant details or superficial information that offer little meaningful insight or practical value.
- **Medium labeled:** This category includes documents that indirectly address the query. While they contain some relevant information, the content is not comprehensive, requiring additional sources for a complete understanding.
- **High labeled:** These documents excel by providing clear, direct, and thorough answers. They present key information in a logical and easy-to-follow way, ensuring accuracy and relevance. As a result, they become valuable resources for the LLM.

Next, both labeled and unlabeled pairs (*query*, *document*) are passed through the embeddings model (e.g., T5-GTR) to generate their respective embeddings. In the third stage,

an autoencoder is employed to map the embeddings from the previous step into a lower-dimensional space (i.e., $\mathbb{R}^{768} \rightarrow \mathbb{R}^9$). This mapping is achieved through a supervised loss function applied to the labeled dataset, combined with a clustering algorithm (e.g., K -Means). A supervised cross-entropy loss is utilized for the labeled data to facilitate the clustering process. Through this weakly supervised process, a dataset of pseudo-labeled data is generated alongside the original labeled data. Ultimately, in the fourth step, this combined dataset is fed into a classification head, whereby a classifier is trained to predict the classes (*Low*, *Medium*, or *High*) based on the input data.

During inference, retrieval evaluators that were achieved in training phase will use various mechanisms and are utilized to evaluate different parts of the retrieved documents. This ensures that the best available parts from the documents are ultimately selected and forwarded to the final model. This process is conducted sequentially by examining the documents at both the passage and sentence levels. In this section, all steps of the process are described step by step.

3.1 Dataset generation

The primary objective of this step is to construct a high-quality dataset for fine-tuning our retrieval evaluators. In practice, two datasets are constructed, each corresponding to a different level of granularity: In one dataset, all documents are of the passage type, while in the other, all documents are of the sentence type. Each of these datasets is ultimately used to train a retrieval evaluator. Based on algorithm 1 Let the dataset be denoted as D . The set of inputs is referred to as X , which consists of multiple pairs of (*query*, *document*) in a special Question-answering Natural Language Inference (QNLI) prompt template, each associated with a label from the set $Y \in \{\text{Low}, \text{Medium}, \text{High}\}$. Initially, a small subset of human-labeled data is considered as the guide dataset, denoted by D' . Correspondingly, the inputs and labels of this dataset are referred to as X' and Y' , respectively. The guide dataset D' is constructed in a balanced manner such that the number of samples for each label category is equal. Another dataset, denoted as D'' , is constructed wherein the inputs X'' are randomly and fairly selected to ensure an equal probability of each label being present. The corresponding label set Y'' is unknown at this stage and must be determined using a clustering algorithm. The final dataset is obtained by combining the guide dataset and the pseudo-labeled dataset:

$$D = D' \cup D'' \quad (1)$$

To evaluate the precision of the clustering algorithm, a test dataset similar in nature to D' is constructed and manually labeled. This dataset, used for evaluation purposes, is denoted as D_{val} . D_{val} originates from a different distribution than D' and, unlike it—which was selected from the training portion of the datasets—has been constructed from the test sections of the original datasets. The algorithm employed to generate Y'' comprises three main stages. First, consider the T5-GTR embedding model as a function denoted by $T5$. The output of this function for each input pair (*query*, *document*) is a vector of dimension 768. Thus, the entire input set X can be transformed into the embedding space using the following relation:

$$T5(X) = E = [e_1, e_2, \dots, e_n] \quad (2)$$

where e_i represents the embedding corresponding to the input pair:

$$e_i = T5(q_i, d_i), \quad e_i \in \mathbb{R}^{768} \quad (3)$$

In the next stage, dimensionality reduction is performed using an autoencoder, which maps the embeddings into a suitable latent space where a better distinction between the *Low*, *Medium*, and *High* classes is preserved. This transformation facilitates the clustering algorithm in accurately grouping the samples.

Algorithm 1 MG-CRAG Training Pipeline

```

1: Input: Subset of Queries  $Q$ , Guide Dataset  $D'$ , Validation Dataset  $D_{\text{val}}$ 
2: Output: Trained T5-GTR model
3: procedure TRAINING
4:   Step 1: Document Retrieval and Pair Creation
5:    $D_{\text{retrieved}} \leftarrow \text{MS\_CONTRIEVER}(Q)$ 
6:   Pairs  $X'' \leftarrow \text{CREATEPAIRSINPROMPTTEMPLATE}(Q, D_{\text{retrieved}})$ 
7:   Step 2: Embedding Generation
8:    $X' \leftarrow \text{GETTEXTS}(D')$ 
9:   Embeddings  $E'' \in \mathbb{R}^{768} \leftarrow \text{T5-GTR-EMBED}(X'')$ 
10:  Embeddings  $E' \in \mathbb{R}^{768} \leftarrow \text{T5-GTR-EMBED}(X')$ 
11:  Step 3: Dataset Generation with Pseudo-Labeling
12:   $L' \leftarrow \text{GETLABELS}(D')$ 
13:   $D'' \leftarrow \text{RANDOMSAMPLE}(X)$ 
14:   $\text{Autoencoder} \leftarrow \text{TRAIN\_AUTOENCODER}(E'', E', L')$ 
15:   $G \leftarrow \text{GETENCODER}(\text{Autoencoder})$ 
16:   $E \leftarrow E' \cup E''$ 
17:   $E_m \in \mathbb{R}^{m \times 9} \leftarrow G(E)$ 
18:  Clusters  $C \leftarrow K\text{-MEANS}(E_m)$ ; Validate on  $D'$  and  $D_{\text{val}}$ 
19:  Cluster into 3 classes (Low, Medium, High) and validate
20:  Pseudo-labels  $Y'' \leftarrow \text{PSEUDOLABEL}(C, D'', D')$ 
21:   $D \leftarrow D' \cup D''$ 
22:  Step 4: Fine-Tuning Retrieval Evaluators
23:  Classification Head  $\leftarrow \text{TRAIN\_CLASSIFIER}(D, E_m)$  to predict quality classes
24: end procedure

```

3.1.1 Latent space projection via autoencoder

Determination of the optimal number of dimensions in the latent space of an autoencoder is critical for effective data representation and dimensionality reduction. To this end, the intrinsic dimension estimation method was utilized to identify the true underlying dimensionality of the data, which is often lower than the observed feature space. Specifically, the Two-Nearest Neighbors (Two-NN) method [46] was employed to estimate the intrinsic dimension of the dataset. This approach relies on the distances to the first and second nearest neighbors of each data point to infer the dimensionality of the underlying manifold. By analyzing the ratio of these distances, a reliable estimate of the dataset's intrinsic structure is provided by the Two-NN method.

In this section, the autoencoder is responsible for dimensionality reduction. The input and output of the autoencoder are the embeddings E , previously obtained in the earlier stage. The autoencoder maps these vectors into a latent space of dimension m .

The encoding and decoding process of the autoencoder can be represented as:

$$G(e) = z, \quad G'(z) = \hat{e} \quad (4)$$

Here, $G(\cdot)$ denotes the encoder, $G'(\cdot)$ the decoder, $e \in E$, the input, $z \in \mathbb{R}^m$ the latent representation, and $\hat{e}$ the reconstruction of the input. To enhance the separability of the

classes (*High*, *Medium*, *Low*) in the latent space, a classification head F based on a softmax layer is applied to the latent representation z to produce predicted labels $\hat{y}$:

$$F(z) = \hat{y} \quad (5)$$

A composite loss function, which combines reconstruction and classification objectives, is used:

$$\mathcal{L}_{\text{recon}}(e, \hat{e}) = \frac{1}{d} \sum_{i=1}^d (e_i - \hat{e}_i)^2 \quad (6)$$

$$\mathcal{L}_{\text{cls}}(y, \hat{y}) = \begin{cases} -\sum_{c=1}^C y_c \log(\hat{y}_c), & \text{if } y \text{ exists} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{cls}} \quad (8)$$

In this formulation, y is the human-provided label (if available), d is the size of dataset, C is the number of classes, and $\hat{y}$ is the predicted label from the classification head. After training, the encoder $G(\cdot)$ can be used to project the entire embedding set E into the latent space of dimension m :

$$G(E) = E_m \quad (9)$$

where $E_m \in \mathbb{R}^m$ denotes the transformed embeddings in the lower-dimensional latent space. See “Appendix B” for more detailed structure of autoencoder.

3.1.2 Clustering unlabeled data

Our framework guides the clustering process using the lower-dimensional embeddings produced by the autoencoder [47]. Using a weakly supervised approach informed by our curated labeled dataset, we employ K -Means clustering to organize documents based on specific criteria. By establishing precise semantic benchmarks, the interpretative capacity of the clustering process is refined, thereby improving the alignment of K -Means assignments with predefined quality levels. The output from the previous step, the latent representation E_m , is used in this stage as the input dataset for clustering. Using unsupervised clustering algorithms, pseudo-labels Y'' are assigned to the data points in E_m , completing the construction of the dataset D'' . The K -Means clustering process partitions the latent space E_m into three clusters, each requiring assignment of one label: *High*, *Medium*, or *Low*. Since part of E_m corresponds to the guide dataset X' , for which the true labels Y' are available. This overlap can be used to evaluate the clustering accuracy and to map clusters to classes. Additionally, the manually labeled test set D_{val} serves as another benchmark for assessing the clustering performance. Finally, only those clustering results are selected for downstream use that demonstrate acceptable average performance across both D' and D_{val} . This ensures that the clustering model generalizes well and can reliably contribute to the construction of a robust training set for the retrieval evaluator.

3.2 Fine-tuning retrieval evaluators

The dataset D , constructed and finalized in the previous steps, is used to train the retrieval evaluators. In this stage, the T5-GTR model weights are frozen, and only a neural network

classification head with residual connections is trained. As the T5-GTR embeddings are pre-computed, they are used directly during training, with updates applied exclusively to the weights of the classification head. The choice of the T5-GTR model for this task was made for several reasons. First, it is based on T5 which ensures conditions similar to those of CRAG. Furthermore, the T5 model has been pre-trained on a similar task named QNLI, which enables it to identify documents containing answers to user queries effectively. Also T5-GTR has less parameters compare to T5. See “Appendix A” for a more detailed discussion of the fine-tuning conditions.

3.3 Multi-granular correction

The multi-granular correction process utilizes two retrieval evaluators, *PLRE* and *SLRE*, arranged sequentially within the system architecture. Initially, the *PLRE* evaluates passages, forwarding high-quality ones to the next stage and, in the absence of high-quality passages, passing medium-quality ones. These passages are then broken down into individual sentences, which the *SLRE* assesses. Based on two inference mechanisms, the selected sentences are ultimately fed to the LLM. This sequential design fulfills three primary objectives; (1) Processing Complex and Lengthy Passages: When the *PLRE* labels a passage as medium quality due to its complexity or length, which hinders accurate evaluation, the *SLRE* extracts relevant content and removes irrelevant sections. This reduces confusion and sharpens the focus of the system. (2) Enhancing High-Quality Passages: Even passages marked as high-quality by the *PLRE* may contain extraneous material that could distract the LLM. The *SLRE* refines these by eliminating irrelevant content, thereby improving overall document quality. (3) Preventing Semantic Inconsistencies: Relying solely on the *SLRE* could lead to misclassification of semantically dependent sentences, where removing one sentence (e.g., labeled medium-quality) might leave the remaining sentence’s meaning incomplete. By combining the *PLRE*’s initial filtering with the *SLRE*’s targeted removal of low-quality content, the system ensures the LLM receives coherent and relevant input.

3.3.1 Inference mechanisms

Three inference mechanisms, (1) Strict Mode, (2) Moderate Mode and (3) Lenient Mode, as presented in Algorithm 2 and depicted in Fig. 2, guide the filtering and selection of sentences for subsequent processing, each applying distinct evaluation criteria to retrieval outputs. Strict Mode, enforces a stringent filtering process, advancing only sentences that receive *High* labels from both retrieval evaluators across different granularity levels to the final stage as final *High* sentences. This rigorous approach lowers the likelihood of sentences achieving a *High* label, thereby increasing dependence on web search results. Sentences tied to passages labeled *Medium* but assigned a *High* label by the *SLRE* are designated as *Medium* final sentences. When no documents achieve a *High* label, these *Medium* sentences are combined with web search results to ensure adequate content availability.

Conversely, moderate mode adopts a more inclusive strategy. Sentences that have received at least one high label from any of the retrieval evaluators are selected as the final high sentences. For the final medium sentences, both retrieval evaluators must have assigned the medium label to the sentence. The lenient mode acts as the least restrictive mechanism for text filtering. In this mode, we will only have the *High* category at the end, with no *Medium* category. Only text segments labeled as *Low* by one of the retrieval evaluators will be removed, while the remaining segments proceed to the next stage. In this case, most of the work falls to

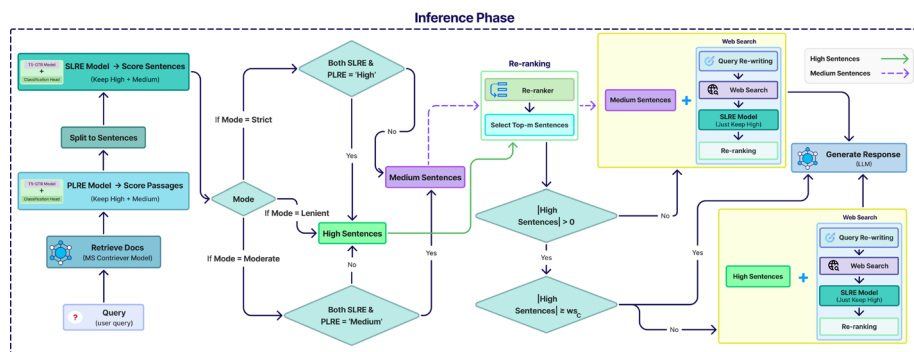


Fig. 2 MG-CRAG Inference Pipeline. It depicts the multi-granular correction process, where retrieved documents are sequentially evaluated using *PLRE* and *SLRE* to filter out low-quality content before it reaches the LLM. The pipeline supports strict, moderate, and lenient modes for flexible quality control, with web search integration to supplement information when high-quality content is insufficient

the re-ranker, and the evaluators have little impact on the filtering process. In the final stage, if we have ws_c number of high sentences, no web search is performed. Otherwise, a web search is carried out. The final sentence selection is as follows:

1. If there is at least one high sentence, only the high sentences are chosen as the final selection.
2. Otherwise, the system falls back to the medium documents, which serve as the backup selection.

Table 1 describes each inference mode briefly.

3.4 Web search methodology

In alignment with the CRAG framework, our methodology employs a web search mechanism to enhance the retrieval process when initial efforts fail to yield high-quality information. As shown in Fig. 2, the parameter ws_c can control the web search rate. The higher the value of this parameter, the more queries utilize web search, and the lower the value, the web search rate decreases. Setting this parameter to 1 makes the algorithm more similar to CRAG, with at least one high sentence, no web search is performed. This section outlines a systematic approach to conducting web searches and refining retrieved data, ensuring that only the most relevant content is provided to the LLM. Web search retrieval is conducted in a manner consistent with CRAG, with results returned as segmented text strips, effectively broken down into individual sentences. The retrieved results undergo a multi-step processing pipeline to ensure quality and relevance:

1. **Sentence Segmentation:** Retrieved passages are segmented into individual sentences, enabling precise evaluation of each unit.
2. **Quality Evaluation with *SLRE*:** Since web search results are returned as segmented text strips consisting of individual sentences, *SLRE* is employed to evaluate their quality. This model assesses each sentence, assigning a quality label—*Low*, *Medium*, or *High*—based on its relevance to the query.
3. **Selection of High-Quality Sentences:** Only sentences labeled *High* are retained, while others are filtered out.

Table 1 Summary of the three inference mechanisms in the MG-CRAG Framework

Mode	Final high sentences	Final medium sentences
Strict Mode	Sentences rated <i>High</i> by both retrieval evaluators	Sentences from <i>Medium</i> passages with <i>High</i> SLRE label
Moderate Mode	Sentences with <i>High</i> label from at least one retrieval evaluator (<i>PLRE</i> or <i>SLRE</i>)	Sentences with <i>Medium</i> label from both retrieval evaluators
Lenient Mode	Sentences with no <i>Low</i> labels from retrieval evaluators	No <i>Medium</i> category exists

4. **Re-ranking:** The re-ranker model reorders the high-quality sentences by scoring their contextual relevance to the query, prioritizing the most pertinent information.
5. **Selection of Top- n Sentences:** The top n sentences are selected from the re-ranked list, with n determined empirically to optimize performance for short-answer datasets (Sect. 4.7).

This process, illustrated in Fig. 2, visually depicts the sequential filtering and refinement of web search results, enhancing comprehension of the methodology.

Algorithm 2 MG-CRAG Inference. the *evaluate_passages(.)* function plays the role of PLRE and the function *evaluate_sentences* plays the role of SLRE.

Require: q (Query), $docs = \{d_1, d_2, \dots, d_k\}$, $mode \in \{strict, moderate, lenient\}$, wsc
Ensure: Selected texts, Web search flag

```

1: (high_docs, medium_docs)  $\leftarrow$  evaluate_passages( $q$ ,  $docs$ )
2: high_sentences  $\leftarrow$  evaluate_sentences( $q$ , high_docs)
3: medium_sentences  $\leftarrow$  evaluate_sentences( $q$ , medium_docs)
4: if  $mode = strict$  then
5:   high_sentences  $\leftarrow$  evaluate_sentences( $q$ , high_docs) where sentences = High
6:   medium_sentences  $\leftarrow$  evaluate_sentences( $q$ , medium_docs) where sentences = are High
7: else if  $mode = moderate$  then
8:   high_sentences  $\leftarrow$  evaluate_sentences( $q$ , high_docs) where sentences  $\neq Low$ 
9:   high_sentences  $\mathrel{+}=$  evaluate_sentences( $q$ , medium_docs) where sentences = High
10:  medium_sentences  $\leftarrow$  evaluate_sentences( $q$ , medium_docs) where sentences = Medium
11: else if  $mode = lenient$  then
12:  high_sentences  $\leftarrow$  evaluate_sentences( $q$ , high_docs) where sentences  $\neq Low$ 
13:  high_sentences  $\mathrel{+}=$  evaluate_sentences( $q$ , medium_docs) where sentences  $\neq Low$ 
14: end if
15: if  $|high\_sentences| > 0$  then
16:   if  $|high\_sentences| \geq wsc$  then
17:     return (high_sentences, 0)
18:   else
19:     return (high_sentences, 1)
20:   end if
21: else
22:   return (medium_sentences, 1)
23: end if

```

4 Experiments

To ensure a fair evaluation and facilitate direct comparison with results from Self-RAG and CRAG studies, most experimental conditions were aligned with those of CRAG. The first phase encompasses the development and training of the *PLRE* and *SLRE* models. Specifically, *PLRE* was trained exclusively on document-level data, while *SLRE* was trained on both document and sentence-level data to enhance its granularity in processing text. The second phase evaluates the performance of the proposed system. The method introduced in this work has been tested solely on short-answer datasets, with a primary focus on improving response accuracy for these datasets.

4.1 Datasets and models

The study employs three datasets: ARC-Challenge, PopQA, and PubHealth, each comprising a training and a test section. The reason behind selecting these datasets is their alignment

and comparability of results with the findings of the CRAG and Self-RAG studies. These datasets represent common points in both works and are presented in short-answer format. The test section of these datasets is sourced directly from the Self-RAG paper, which includes passages retrieved by the *MS-Contriever* model. These pre-retrieved passages are then utilized directly in testing phase to maintain uniformity and comparability with the CRAG framework (CRAG also uses the same retrievals from Self-RAG). To construct D' , 180 passages and 180 sentences were manually annotated, evenly distributed across the three datasets. This resulted in a human-labeled dataset containing 180 passage-level and 180 sentence-level annotations. To build D'' , 200 queries were randomly sampled from the training sections of each dataset (PopQA, PubHealth, and ARC-Challenge), totaling 600 queries. For each query, all retrieved contexts associated with it were included to ensure a distributed representation of High, Medium, and Low labels across the data. This approach helps to have query-context pairs with diverse qualities. The query-context pairs were formatted into QNLI prompt template (available in “Appendix A”) to be usable in next steps.

The classification head, implemented as a light residual network, is trained on D to predict the quality of the document at two granularity levels: *PLRE* for passages and *SLRE* for sentences. Freezing the T5-GTR embeddings ensures efficient training while preserving semantic relevance. The T5-GTR model, used in our work as part of the pre-trained retrieval evaluators, has 335 M parameters, making it significantly lighter compared to the original T5 model utilized in CRAG, which contains 738 M parameters. In our approach, a neural network was only trained as a classification head, which is far less computationally expensive than fine-tuning the T5-Large model. Furthermore, due to the reduced number of parameters, the response speed of this model is substantially faster compared to T5-Large. The decision to use a residual network as the classification head was based on a trial-and-error approach, where various architectures, including Feed Forward networks and Transformer-based networks, have been tested. Compared to these alternatives, the residual network exhibited better power and effectiveness.

Finally, the SelfRAG-LLaMA2-7b model, a fine-tuned large language model, was employed (which has been used in both CRAG [6] and Self-RAG [5]) for the experiments to achieve exact same conditions with the baselines.

4.1.1 Guide set preparation and annotation process

To construct the guide set, a single annotator was tasked with assigning the appropriate labels. For the selection of passages labeled as High, the annotator examined randomly sampled question indices from all three datasets. The corresponding contexts were reviewed, and passages that clearly and sufficiently contained the correct answer were designated as High. For the Medium label, the annotator selected passages associated with randomly chosen questions that either indirectly contained the answer or provided supporting information relevant to deriving the correct response, but did not explicitly include it. Finally, passages that were irrelevant to the question or provided no meaningful contribution toward answering it were assigned the Low label. The same annotation procedure was applied at the sentence level, where sentences exhibiting equivalent characteristics were labeled accordingly. The entire annotation process was completed by a single annotator over approximately 12 h.

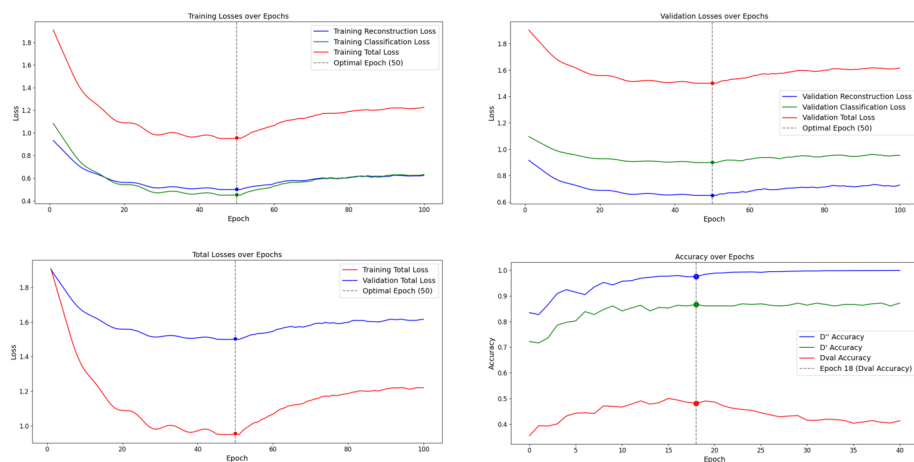


Fig. 3 Training, validation, total losses, and accuracy over epochs

4.2 Autoencoder training

The autoencoder architecture employs a structure that includes dense and batch normalization layers with dropouts and the ReLU activation function. Our experimental results identified an intrinsic dimension of 9 ($m = 9$) for the latent space. The autoencoder was trained using both the dataset D' and the dataset D'' to reduce the dimensionality of the input data from 768 features to 9 features, allowing efficient representation learning. Based on the plots presented in Fig. 3, the loss and accuracy for both training phase of the autoencoder and the classification head are shown across epochs. These plots were utilized to select the optimal values at each stage. Specifically, the number of epochs was determined based on the validation loss and validation accuracy. In fact, the validation data were leveraged to fine-tune hyper-parameters such as the number of epochs (available in “Appendix B”).

4.3 Clustering and training of the classification head

For clustering, the latent representations generated by the autoencoder (Sect. 3.1.1) are partitioned using the K -Means algorithm into three clusters corresponding to *High*, *Medium*, and *Low* quality labels. This weakly supervised approach leverages the guide dataset D' to align cluster assignments with human-annotated quality categories, ensuring semantic consistency. The clustering accuracy is validated against both D' and the manually labeled validation set D_{val} , guaranteeing robustness across seen and unseen data. After clustering process, pseudo-labels for D'' are generated and combined with D' to form the final training dataset D . As shown in Table 2, the clustering and classification accuracies for both *PLRE* and *SLRE* are evaluated across D' and D_{val} . The results demonstrate strong alignment between pseudo-labels and ground-truth annotations, particularly for passage-level processing, validating the effectiveness of the autoencoder-guided clustering. Notably, the *SLRE* achieves competitive accuracy on sentence-level data despite shorter text spans, showing its ability to filter fine-grained noise. These outcomes align with CRAG’s reported benchmarks, confirming that the proposed weakly supervised framework retains CRAG’s efficacy while reducing reliance on

Table 2 Clustering and classification accuracies for passage and sentence levels

Dataset	Granularity	Clustering ^a (Accuracy %)	Classification ^b (Accuracy %)
D'	Passage	37.44	88.33
D'	Sentence	41.76	90.55
D_{val}	Passage	44.44	35.97
D_{val}	Sentence	46.11	40.21

^aClustering Accuracy refers to the accuracy of K -Means clustering

^bClassification Accuracy refers to the accuracy of the classification head after fine-tuning

heuristic labels. The residual network architecture further enhances discriminative power, enabling precise stratification of document quality.

The results in Table 3 highlight several important observations that explain some important notes:

- **Clustering Accuracy and Pseudo-Labels:** The clustering accuracies reported in Table 2 correspond to pseudo-labels generated during the weakly supervised stage. Given the presence of three classes (with a random baseline of 33.3%), the obtained accuracy is reasonable. However, pseudo-labels inherently contain noise, which contributes to reduced precision when compared to gold-labeled data.
- **Challenge of Ambiguity Detection:** Identifying ambiguous samples is inherently difficult, and even human annotators often disagree on such cases. The T5-GTR model, therefore, struggles to effectively capture ambiguity using its embeddings. Our analysis shows that the model has difficulty separating *medium-high* and *medium-low* classes, though it performs well in distinguishing between *high* and *low* relevance samples. Table 3 shows that by merging the medium and high classes, the precision, recall, and F1-score on D_{val} increase, resulting in a higher overall classification accuracy. The T5-GTR model was selected to ensure comparability with the CRAG framework. Nevertheless, the proposed MG-CRAG framework is model agnostic and can incorporate stronger embedding models that may better capture ambiguous decision boundaries.
- **Gap Between D' and D_{val} :** The classification accuracies obtained on the supervised D' dataset are higher because the model is trained and evaluated on data drawn from the same distribution. In contrast, the D_{val} dataset represents a different, unseen distribution with gold human-labeled samples. As a result, the model's performance naturally drops when evaluated on D_{val} .
- **Effect of Dataset Size:** Both D' and D_{val} are considerably smaller in size than the unlabeled dataset. This limited sample size increases the variance of evaluation metrics and can exaggerate accuracy differences between datasets.
- **Inference Mechanisms and Robustness:** To mitigate ambiguity detection challenges, we designed three inference mechanisms—*Strict*, *Moderate*, and *Lenient*—supported by two evaluators, **PLRE** and **SLRE**. These mechanisms enhance robustness and adaptability across distributions, representing one of the key strengths of the MG-CRAG framework.
- **Better SLRE Performance on Passages:** According to the results presented in Table 3, it can be observed that the accuracy of SLRE on passages is higher than that of PLRE. One possible reason for this is that SLRE has been trained on passages as well. The motivation behind this training is the need to analyze longer texts—beyond single sentences—in the web search results, a task that must be handled by SLRE. Despite this higher accuracy,

Table 3 Performance of PLRE and SLRE on human-labeled question–context pairs

Evaluator	Granularity	Dataset	Precision	Recall	F1-Score
PLRE	Passage	D'	0.967	0.983	0.974
PLRE	Passage	D_{val}	0.644	0.850	0.733
PLRE	Sentence	D'	0.712	0.948	0.813
PLRE	Sentence	D_{val}	0.639	0.825	0.720
SLRE	Passage	D'	0.991	0.991	0.991
SLRE	Passage	D_{val}	0.671	0.885	0.763
SLRE	Sentence	D'	0.966	0.966	0.966
SLRE	Sentence	D_{val}	0.681	0.836	0.751

For evaluation, the *High* and *Medium* classes were merged into a single positive category, enabling binary classification. This formulation highlights that reduced clustering accuracy primarily arises from ambiguity detection challenge

Table 4 Comparing the accuracy of different re-ranker models on each dataset

Re-Ranker	ARC (Accuracy %)	Pub (Accuracy %)	PopQA (Accuracy %)
MPNet	68.85	74.37	59.33
Self-RAG score	67.57	75.58	52.68
RoBERTa	68.08	74.56	59.89

The best values are highlighted in boldface for each column

when examining the final outcomes, we found that replacing PLRE with SLRE leads to a decrease in accuracy for the ARC and PubHealth datasets. However, in the PopQA dataset, a single model was used instead of separate PLRE and SLRE models. Another possible reason for this behavior is the limited number of samples in the human-labeled datasets, which may not adequately evaluate the generalization capability of the model.

4.4 Re-ranker selection

Experiments revealed that for each dataset, a specific re-ranker model yields the best results. The MPNet model with 100 M parameters was used for the ARC-Challenge dataset, RoBERTa model for the PopQA dataset, and the Self-RAG scores for the Pubhealth dataset, as summarized in Table 4. For each dataset, the re-ranker was selected empirically based on validation performance. Several widely used re-ranker models (e.g., RoBERTa, MPNet, and T5-GTR) were tested and the model that yielded the best validation accuracy for that specific dataset was chosen.

4.5 Effect of inference mechanisms on accuracy

To evaluate the efficacy of three inference mechanisms—Strict, Moderate, and Lenient—we assessed their accuracy across three diverse datasets, as shown in Table 5, finding that the Moderate mechanism achieved the highest accuracy on ARC-Challenge, the Strict mechanism performed best on PubHealth, and the Lenient mechanism yielded the top accuracy on PopQA. On the PopQA dataset, since SLRE has also been trained on passages, it was used

Table 5 Accuracy comparison of Strict Mode, Moderate Mode, and Lenient Mode on ARC-Challenge, PubHealth, and PopQA datasets

Mode	ARC (Accuracy %)	Pub (Accuracy %)	PopQA (Accuracy %)
Strict	66.80	75.58	58.82
Moderate	68.85	72.94	58.90
Lenient	68.26	72.94	59.89

The best values are highlighted in boldface for each column

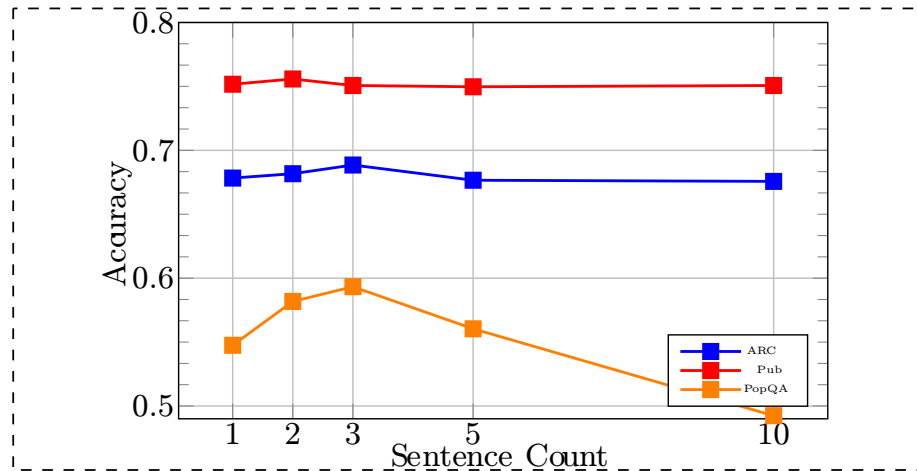


Fig. 4 Impact of selecting different numbers of sentences on accuracy in various datasets. The chart shows how evaluation accuracy is affected by the number of sentences chosen. Selecting too few sentences leads to the loss of important information, while selecting too many sentences causes model confusion. It is suggested to use 2 or 3 sentences

as PLRE as well, because the original PLRE did not perform well on this dataset. In other words, for the PopQA dataset, a single model was used in both the PLRE and SLRE roles (Fig. 4).

4.6 Results

The effectiveness of the proposed MG-CRAG framework is evaluated through experiments on three benchmark datasets: ARC-Challenge, PubHealth, and PopQA. As reported in Table 6, experiments show that the proposed method achieves higher accuracy compared to the Self-RAG method across all datasets. The framework also achieves state-of-the-art performance on the ARC-Challenge dataset, attaining an accuracy of 68.85%, which outperforms existing methods. This enhancement underscores the efficacy of weakly supervised pseudo-labeling and multi-granular document evaluation in improving response accuracy. In the PubHealth dataset experiments, the proposed model achieved an accuracy equal to that of CRAG 75.58%. For the PubHealth dataset, superior results were obtained when SLRE was not applied to the web search outputs. However, applying SLRE on web search contents reduced the accuracy to 74.97%. The reduction in accuracy accompanying the application of SLRE may be attributed to domain shift, inasmuch as web search results exhibit considerable diversity, whereas the

Table 6 Accuracy comparison of MG-CRAG against baseline methods

Method (or LLM)	ARC (Accuracy %)	Pub (Accuracy %)	PopQA (Accuracy %)
Other LLMs without retrieval			
Llama2-7b	21.8	34.2	14.7
Alpaca-7b	45.0	49.8	23.6
Llama2-13b	29.4	29.4	14.7
Alpaca-13b	54.9	55.5	24.4
Other LLMs with retrieval			
Llama2-7b	48.0	30.0	38.2
Alpaca-7b	48.0	40.2	46.7
Llama2-13b	26.0	30.2	45.7
Alpaca-13b	57.6	51.1	46.1
SelfRAG-LLaMA2-7b			
w/o RAG	66.97	70.61	30.45
RAG*	53.22	39.04	52.82
RAG + Sentence Re-ranker	66.63	70.41	51.75
RAG + Passage Re-ranker	57.42	47.92	55.54
CRAG*	68.60	75.58	59.82
Self-RAG*	67.32	72.44	54.89
MG-CRAG	68.85	75.58	59.89

The best values in each column are highlighted in bold. The same SelfRAG-LLaMA2-7b model is the used LLM to obtain comparable results. The prompts have also been directly adopted without modifications from the CRAG work

*The reported accuracies for these cases have been extended to two decimal places based on the number of correct responses. In cases where two values are possible, the larger value has been considered

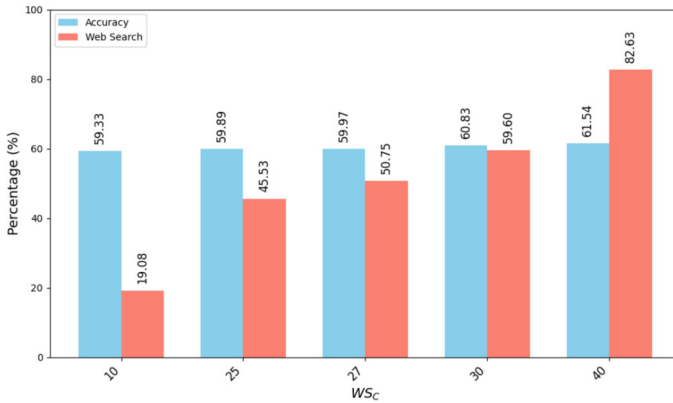


Fig. 5 Impact of the ws_c parameter on web search rate and accuracy on PopQA dataset

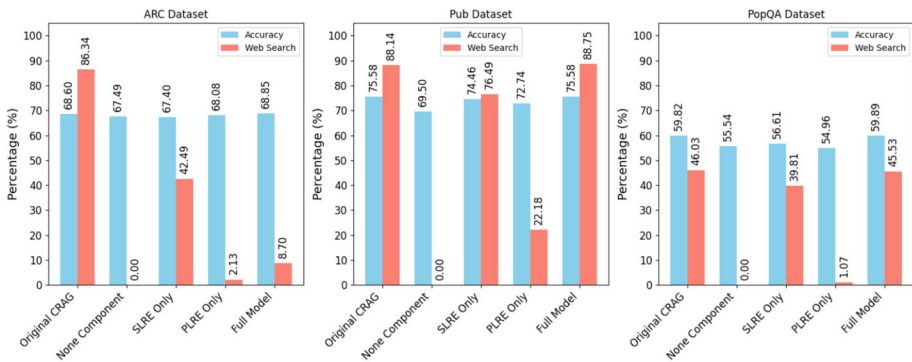


Fig. 6 Performance and web search rate comparison across ARC-challenge, PubHealth, and PopQA datasets with different module removals. The chart of each dataset shows that the combination of *PLRE* and *SLRE* performs better than using either one alone. Also, the impact of removing each of these two modules on reducing accuracy and web search rate is demonstrated

training data for SLRE were derived from a dataset that did not encompass web search results. For PubHealth, the top 5 web search documents (not split into sentences, although they are as short as one to three sentences, corresponding to the strips in CRAG) are used. On the PopQA dataset—where CRAG’s retrieval evaluator was specifically trained—MG-CRAG achieves higher accuracy (59.89%) while further reducing the web search rate compared to CRAG. Experiments show that by adjusting the parameter ws_c and increasing the web search rate, as illustrated in Fig. 5, our method has achieved higher accuracy (61.54%). In all reported accuracies, MG-CRAG achieved a lower web search rate compared to CRAG, as shown in Fig. 6.

4.7 Impact of the number of sentences on performance

For short-answer datasets, an evaluation was conducted to determine the accuracy achieved based on varying sentence counts. The results are presented in Fig. 4, which details the accuracy corresponding to 1, 2, 3, 5, and 10 sentences. Additionally, an analysis was performed to assess the effect of these specific sentence counts on performance. The study investigates

how the number of sentences influences the accuracy and identifies which sentence count yields the highest accuracy. The graph, shown in Fig. 4, illustrates the relationship between the number of sentences and the corresponding accuracy for a short-answer datasets.

4.8 Ablation study

The primary objective of the experiments in this section is to evaluate how the combination of two evaluators improves the quality of the response and to assess the impact of performing evaluations at a single level of granularity. Consistent with CRAG, ChatGPT–3.5-turbo was utilized as the query re-writer, contributing to competitive performance despite differences in training data. Based on the results in Fig. 6, combining the two evaluators significantly increases accuracy for all datasets. This finding suggests that *PLRE* may eliminate some critical documents or it can allow all parts of a document which can be irrelevant, leading to the loss of information necessary for answering questions. Additionally, using *SLRE* alone can lead to the removal of important parts of sentences in cases where sentences are sequential and do not convey complete meaning individually and the accuracy is lower compared to the combination of *PLRE* and *SLRE*. The Baseline (No *SLRE/PLRE*) scenario involves only the re-ranker, producing results purely through re-ranking, without incorporating *SLRE* or *PLRE* models. Therefore, the baseline scenario does not use web search. The analysis aims to clarify how each component contributes to the final results in terms of accuracy and web search reduction.

5 Conclusion

MG-CRAG was presented in this study, a novel framework that extends the CRAG approach through multi-granular document evaluation and weakly supervised pseudo-labeling. Our key contributions include: (1) a four-stage training pipeline combining manual annotation with autoencoder-guided clustering for robust label generation, (2) a sequential evaluation architecture using both passage-level and sentence-level retrieval evaluators, and (3) multiple inference mechanisms tailored for different quality requirements. Experimental results demonstrate that MG-CRAG achieves superior performance on the ARC-Challenge and PopQA datasets while maintaining competitive accuracy on PubHealth benchmarks. The effectiveness of this framework is particularly notable given its reduced parameter count (352 M+100 M vs. CRAG's 738 M) and faster inference speed.

The ablation studies reveal important insights about granularity selection, showing that while the combined *PLRE+SLRE* approach generally improves accuracy. Our analysis of sentence count impact provides practical guidance for implementation, suggesting 2–3 sentences as optimal for short-answer generation. The ability of framework to reduce web search dependency (8.7% web search percentage for ARC-Challenge) while maintaining accuracy demonstrates its effectiveness in quality-aware filtering.

6 Future work

Future work could explore dynamic granularity selection, where the system automatically determines the optimal evaluation level based on query characteristics. Additionally, extending the framework to handle long-form answers and investigating the integration of larger

language models present promising directions for further improving retrieval-augmented generation systems. In this study, the MG-CRAG framework was evaluated on short-answer datasets. To extend the scope of this research, MG-CRAG can be applied to long-answer datasets, as the framework imposes no intrinsic restrictions that would prevent such generalization. A key distinction between long-answer and short-answer datasets lies in their underlying objectives. Long-answer datasets primarily aim to filter out irrelevant content, requiring the aggregation of multiple retrieved passages to generate coherent, extended responses. In contrast, short-answer datasets typically rely on identifying a single sentence that contains the correct answer, which is sufficient to produce an accurate result.

Consider a scenario where several passages are retrieved for a long-answer dataset (such as Bio). Some passages may pertain to entities with names similar to the target entity but refer to different subjects. The multi-granular evaluation process in MG-CRAG ensures that such irrelevant passages are comprehensively removed by the PLRE. The remaining passages that pass this stage possess higher reliability, enabling the SLRE to apply less stringent filtering criteria.

Consequently, it is crucial to select the inference mechanism in accordance with dataset characteristics, as this flexibility represents one of the principal strengths of the MG-CRAG framework. By increasing the number of classification levels and introducing additional filtering mechanisms, the degree of strictness within MG-CRAG can be finely controlled.

Furthermore, MG-CRAG is independent of the retrieval model. All its processing is performed post-retrieval, implying that substituting the base retriever (e.g., replacing MS-Contriever with another retrieval model) merely alters the retrieved passages, while the MG-CRAG framework remains applicable. Nevertheless, the framework is sensitive to the chunk size used during retrieval. When chunk sizes encompass multiple passages, coarser granularities such as topic-level_evaluation can be incorporated. Conversely, if the chunk size is reduced to the sentence level, finer granularities such as proposition-level evaluation—as proposed in prior studies (e.g., [7])—can be integrated to enhance precision.

Appendix

A Fine-tuning details

This section explains the fine-tuning process of the PLRE and SLRE. The complete fine-tuning conditions, hyperparameter selections, and detailed model architecture are provided below. For further information on implementation, please refer to the code on GitHub.

A.1 Fine-tuning setup

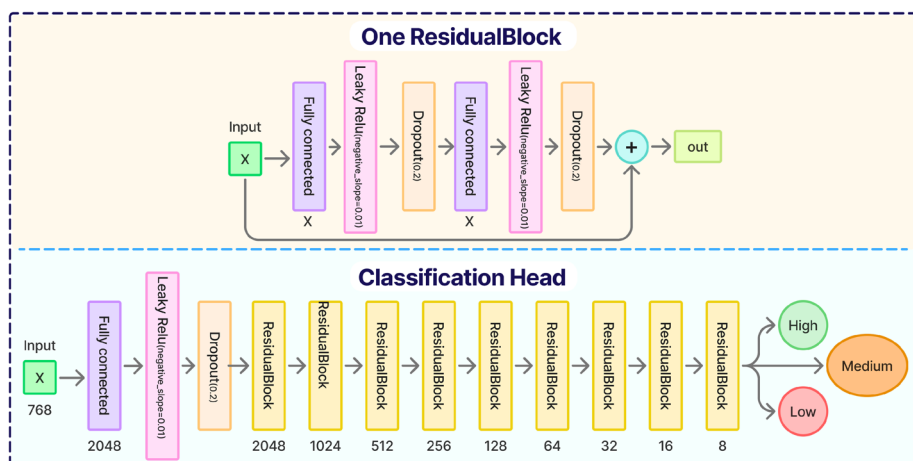
The fine-tuning process was conducted under standardized experimental conditions to ensure reproducibility. Table 7 summarizes the complete set of fine-tuning hyper-parameters, including learning rate, batch size, and the number of epochs. These values were selected based on empirical validation to achieve a balance between convergence stability and generalization performance. The training process of classification head has been continued for 50 epochs and has been saved in the end of each epoch. Then the best model based on the accuracy of the validation set was chosen.

Table 7 Fine-tuning hyper-parameters and their values

Parameter	Value
Learning rate	0.001
Minimum learning rate	1e−6
Scheduler	Reduce LR on plateau
Scheduler Patience	5
Scheduler Factor	0.5
Batch size	10
Epochs	14 for PLRE, 18 for SLRE
Optimizer	Adam
Weight decay	0
Dropout rate	0.2

A.2 Model architecture

The classification head employed in this study follows a fully connected feed-forward design enhanced with residual connections to improve gradient flow and model stability during fine-tuning. The model begins with an input projection layer that maps the input vector of length 768 (output size of T5-GTR) to a 2048-dimensional latent space, followed by a dropout layer with a rate of 0.2 to prevent overfitting. The core of the network consists of a cascade of nine residual blocks, each comprising two linear transformations with Leaky ReLU activations (`negative_slope = 0.01`) and dropout regularization. Whenever the input and output dimensions differ, a linear down-sampling layer aligns the residual connection, ensuring dimensional consistency across skip connections. The complete model architecture diagram is shown in Fig. 7.

**Fig. 7** Classification head structure composed of multiple residual blocks

A.3 Special prompt

The related questions and contents were provided to the T5-GTR model in the form of a specific prompt. This prompt format was originally part of the original T5 model's pre-training data (QNLI). The prompt is structured as follows:

qnli question: {question here} sentence: {context here}

B Autoencoder details

This section discusses the exact autoencoder training setup and architecture of the model. All training hyper-parameters are found in Table 8. The neural structure of autoencoder is also indicated in Fig. 8.

Table 8 Autoencoder training hyper-parameters and their values

Parameter	Value
Learning rate	0.001
Minimum learning rate	0
Scheduler	Reduce LR on plateau
Scheduler Patience	3
Scheduler Factor	0.5
Batch size	1000
Epochs	50
Optimizer	Adam
Weight decay	0
Reconstruction factor	0.53
Classification factor	0.47

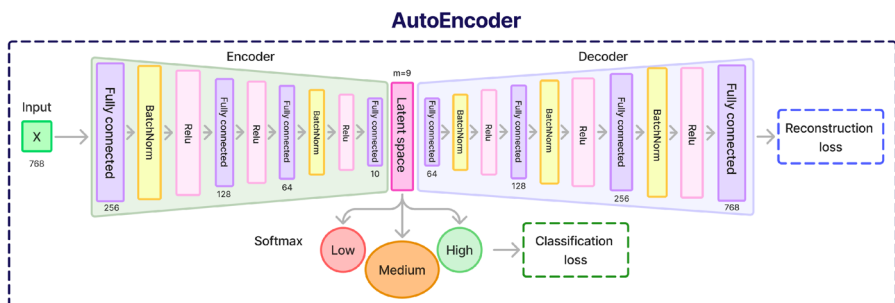


Fig. 8 Autoencoder architecture with reconstruction and classification losses

Author Contributions Negin Masoumi involved in conceptualization, formal analysis, investigation, methodology, software, validation, visualization, writing—original draft, writing—review & editing. Omid Davar took part in conceptualization, formal analysis, investigation, methodology, software, validation, visualization, writing—original draft, writing—review & editing. Mahdi Eftekhari involved in conceptualization, formal analysis, investigation, methodology, project administration, resources, supervision, validation, visualization, writing—review & editing.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Funding The authors declare that no funds, grants, or other support was received during the preparation of this manuscript.

Conflict of interest The authors declare no conflict of interest.

References

1. Xu Z, Jain S, Kankanhalli M (2024) Hallucination is inevitable: an innate limitation of large language models. [arXiv:2401.11817](#)
2. Huang L, Yu W, Ma W, Zhong W, Feng Z, Wang H, Chen Q, Peng W, Feng X, Qin B, Liu T (2025) A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions. *ACM Trans Inf Syst* 43(2):1–55. <https://doi.org/10.1145/3703155>
3. Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W-t, Rocktäschel T, Riedel S, Kiela D (2021) Retrieval-augmented generation for knowledge-intensive NLP tasks. [arXiv:2005.11401](#)
4. Gao Y, Xiong Y, Gao X, Jia K, Pan J, Bi Y, Dai Y, Sun J, Wang M, Wang H (2024) Retrieval-augmented generation for large language models: a survey. [arXiv:2312.10997](#)
5. Asai A, Wu Z, Wang Y, Sil A, Hajishirzi H (2023) Self-RAG: learning to retrieve, generate, and critique through self-reflection. [arXiv:2310.11511](#)
6. Yan S-Q, Gu J-C, Zhu Y, Ling Z-H (2024) Corrective retrieval augmented generation. [arXiv:2401.15884](#)
7. Chen T, Wang H, Chen S, Yu W, Ma K, Zhao X, Zhang H, Yu D (2024) Dense X retrieval: what retrieval granularity should we use?. [arXiv:2312.06648](#)
8. Holtzman A, Buys J, Du L, Forbes M, Choi Y (2020) The curious case of neural text degeneration. [arXiv:1904.09751](#)
9. Tonmoy SMTI, Zaman SMM, Jain V, Rani A, Rawte V, Chadha A, Das A (2024) A comprehensive survey of hallucination mitigation techniques in large language models. [arXiv:2401.01313](#)
10. Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P (2023) Survey of hallucination in natural language generation. *ACM Comput Surv* 55(12):1–38. <https://doi.org/10.1145/3571730>
11. Bubeck S, Chandrasekaran V, Eldan R, Gehrke J, Horvitz E, Kamar E, Lee P, Lee YT, Li Y, Lundberg S, Nori H, Palangi H, Ribeiro MT, Zhang Y (2023) Sparks of artificial general intelligence: early experiments with GPT-4. [arXiv:2303.12712](#)
12. Tang Z, Chatterjee R, Garg S (2025) Mitigating hallucinated translations in large language models with hallucination-focused preference optimization. [arXiv:2501.17295](#)
13. Wen, X Lu X, Guan X, Lu Y, Lin H, He B, Han X, Sun L (2024) On-policy fine-grained knowledge feedback for hallucination mitigation. [arXiv:2406.12221](#)
14. Kim M, Kim M, Bae J, Choi S, Kim S, Chang B (2024) ESREAL: exploiting semantic reconstruction to mitigate hallucinations in vision-language models. [arXiv:2403.16167](#)
15. Wang T, Liu Y, Liang JC, Zhao Cui Y, Mao Y, Nie S, Liu J, Feng F, Xu Z, Han C, Huang L, Wang Q, Liu D (2024) M²PT: multimodal prompt tuning for zero-shot instruction learning. [arXiv:2409.15657](#)
16. Kim J, Lim J (2025) Strong consistency of sparse K-means clustering. [arXiv:2501.09983](#)
17. Guo W, Lin K, Ye W (2021) Deep embedded K-means clustering. [arXiv:2109.15149](#)
18. Ma Y, He H, Lei Z, Niu Z (2024) Masked AutoEncoder for graph clustering without pre-defined cluster number k. [arXiv:2401.04741](#)
19. Zheng C, Zhang K, Wang HJ, Fan L, Wang Z (2021) Enhanced Seq2Seq autoencoder via contrastive learning for abstractive text summarization. [arXiv:2108.11992](#)

20. Ge P, Ren C-X, Dai D-Q, Feng J, Yan S (2020) Dual adversarial autoencoders for clustering. *IEEE Trans Neural Netw Learn Syst* 31(4):1417–1424. <https://doi.org/10.1109/tnnls.2019.2919948>
21. Weng X, An J, Ma X, Qi B, Luo J, Yang X, Dong JS, Huang L (2025) Clustering properties of self-supervised learning. [arXiv:2501.18452](https://arxiv.org/abs/2501.18452)
22. Wu X, Varshney LR (2024) Transformer-based causal language models perform clustering. [arXiv:2402.12151](https://arxiv.org/abs/2402.12151)
23. Mersha MA, yigezu MG, Kalita J (2024) Semantic-driven topic modeling using transformer-based embeddings and clustering algorithms. [arXiv:2410.00134](https://arxiv.org/abs/2410.00134)
24. Li W, Liu K, Zhang X, Lei X, Ma W, Liu Y (2025) Efficient dynamic clustering-based document compression for retrieval-augmented-generation. [arXiv:2504.03165](https://arxiv.org/abs/2504.03165)
25. Diaz-Rodriguez J (2025) k-LLMmeans: scalable, stable, and interpretable text clustering via LLM-based centroids. [arXiv:2502.09667](https://arxiv.org/abs/2502.09667)
26. Hu L, Jiang M, Dong J, Liu X, He Z (2024) Interpretable clustering: a survey. [arXiv:2409.00743](https://arxiv.org/abs/2409.00743)
27. Starosta B, Kłopotek MA, Wierzchoń ST (2023) Explainable graph spectral clustering of text documents. [arXiv:2308.00504](https://arxiv.org/abs/2308.00504)
28. Chen, B, Rouditchenko A, Duarte K, Kuehne H, Thomas S, Boggust A, Panda R, Kingsbury B, Feris R, Harwath D, Glass J, Picheny M, Chang S-F (2021) Multimodal clustering networks for self-supervised learning from unlabeled videos. [arXiv:2104.12671](https://arxiv.org/abs/2104.12671)
29. Yu Y, Ping W, Liu Z, Wang B, You J, Zhang C, Shoenybi M, Catanzaro B (2024) RankRAG: unifying context ranking with retrieval-augmented generation in LLMs. [arXiv:2407.02485](https://arxiv.org/abs/2407.02485)
30. Chan C-M, Xu C, Yuan R, Luo H, Xue W, Guo Y, Fu J (2024) RQ-RAG: learning to refine queries for retrieval augmented generation. [arXiv:2404.00610](https://arxiv.org/abs/2404.00610)
31. Fan T, Wang J, Ren X, Huang C (2025) MiniRAG: towards extremely simple retrieval-augmented generation. [arXiv:2501.06713](https://arxiv.org/abs/2501.06713)
32. Han H, Wang Y, Shomer H, Guo K, Ding J, Lei Y, Halappanavar M, Rossi RA, Mukherjee S, Tang X, He Q, Hua Z, Long B, Zhao T, Shah N, Javari A, Xia Y, Tang J (2025) Retrieval-augmented generation with graphs (GraphRAG). [arXiv:2501.00309](https://arxiv.org/abs/2501.00309)
33. Liang X, Niu S, Li Z, Zhang S, Wang H, Xiong F, Fan JZ, Tang B, Song S, Wang M, Yang J (2025) SafeRAG: benchmarking security in retrieval-augmented generation of large language model. [arXiv:2501.18636](https://arxiv.org/abs/2501.18636)
34. Singh A, Ehtesham A, Kumar S, Khoei TT (2025) Agentic retrieval-augmented generation: a survey on agentic RAG. [arXiv:2501.09136](https://arxiv.org/abs/2501.09136)
35. Zhou H, Lee K-H, Zhan Z, Chen Y, Li Z, Wang Z, Haddadi H, Yilmaz E (2025) TrustRAG: enhancing robustness and trustworthiness in RAG. [arXiv:2501.00879](https://arxiv.org/abs/2501.00879)
36. Sairaj RT, Balasundaram SR (2025) Ensemble learning with rag model to reduce redundant question topics in auto-generated exam questions. *Discover Comput* 28:1–17. <https://doi.org/10.1007/s10791-025-09683-2>
37. Sairaj RT, Balasundaram SR (2025) Ontology mapping for retrieval augmented modelling to reduce factual hallucinations in pre-trained language model-based auto-generated questions. *Appl Ontol*. <https://doi.org/10.1177/15705838251343009>
38. Liang M, Arun A, Wu Z, Munoz C, Lutch J, Kazim E, Koshiyama A, Treleaven P (2024) THaMES: an end-to-end tool for hallucination mitigation and evaluation in large language models. [arXiv:2409.11353](https://arxiv.org/abs/2409.11353)
39. Sharma K, Kumar P, Li Y (2024) OG-RAG: ontology-grounded retrieval-augmented generation for large language models. [arXiv:2412.15235](https://arxiv.org/abs/2412.15235)
40. Park Y, Witherell P, Surovi NA, Cho H (2024) Ontology-based Retrieval Augmented Generation (RAG) for GenAI-supported Additive Manufacturing. National Institute of Standards and Technology (NIST). Accessed 21 Oct 2025. https://www.nist.gov/publications/ontology-based-retrieval-augmented-generation-rag-genai-supported-additive?utm_source=chatgpt.com
41. Toro S, Anagnostopoulos AV, Bello SM, Blumberg K, Cameron R, Carmody L, Diehl AD, Dooley DM, Duncan WD, Fey P, Gaudet P, Harris NL, Joachimiak MP, Kiani L, Lubiana T, Munoz-Torres MC, O’Neil S, Osumi-Sutherland D, Puig-Barbe A, Reese JT, Reiser L, Robb SMC, Ruemping T, Seager J, Sid E, Stefancsik R, Weber M, Wood V, Haendel MA, Mungall CJ (2024) Dynamic retrieval augmented generation of ontologies using artificial intelligence (dragon-AI). *J Biomed Semant* 15. <https://doi.org/10.1186/s13326-024-00320-3>
42. Giglou HB, D’Souza J, Auer S (2024) LLMs4OL 2024 overview: the 1st large language models for ontology learning challenge. [arXiv:2409.10146](https://arxiv.org/abs/2409.10146)
43. Nguyen L, Barcelos E, French R, Wu Y (2025) KROMA: ontology matching with knowledge retrieval and large language models. [arXiv:2507.14032](https://arxiv.org/abs/2507.14032)
44. Bergeron L, Buhnla I, François J, State R (2025) HalluGuard: evidence-grounded small reasoning models to mitigate hallucinations in retrieval-augmented generation. [arXiv:2510.00880](https://arxiv.org/abs/2510.00880)

45. Choubey PK, Su X, Luo M, Peng X, Xiong C, Le T, Rosenman S, Lal V, Mui P, Ho R, Howard P, Wu C-S (2024) Distill-SynthKG: distilling knowledge graph synthesis workflow for improved coverage and efficiency. [arXiv:2410.16597](https://arxiv.org/abs/2410.16597)
46. Facco E, d'Errico M, Rodriguez A, Laio A (2017) Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Sci Rep* 7(1). <https://doi.org/10.1038/s41598-017-11873-y>
47. Lu S, Li R (2021) DAC: deep autoencoder-based clustering, a general deep learning framework of representation learning. [arXiv:2102.07472](https://arxiv.org/abs/2102.07472)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



effectively in diverse environments.

Negin Masoumi is a versatile professional bridging AI research, front-end development, and UI/UX design. She earned her M.Sc. in Artificial Intelligence and her B.Sc. in Computer Engineering from Shahid Bahonar University of Kerman. She was born in Kerman, Iran. In the realm of AI, Negin's research emphasizes Machine Learning, with a particular focus on Natural Language Processing (NLP) and Large Language Models (LLMs). Currently, she is engaged in pioneering AI systems capable of autonomously designing applications, striving to create scalable solutions that enhance the intuitiveness and efficiency of human-computer interactions. In addition to her research endeavors, Negin is a skilled frontend developer and UI/UX designer. She excels at transforming complex AI functionalities into seamless, user-friendly digital experiences utilizing modern web technologies. Her technical expertise encompasses Python for AI innovation, along with a robust proficiency in contemporary frontend frameworks and tools. Negin is also fluent in both Persian and English, allowing her to communicate



Omid Davar is an AI Researcher who was born in Kerman, Iran and received his M.Sc. in Artificial Intelligence and his B.Sc. in Computer Engineering from Shahid Bahonar University of Kerman. With a decade of professional experience as a web developer, he seamlessly bridges the gap between software engineering and advanced machine learning. His primary research and professional interests lie in Machine Learning and Artificial Intelligence, with a specialized focus on Natural Language Processing (NLP), extensive hands-on experience with Large Language Models (LLMs), and Graph Neural Networks (GNNs). Driven by a vision to build scalable AI solutions that improve human-computer interaction, he is also deeply interested in the emerging field of Agentic AI. Highly proficient across multiple technological stacks, his core programming languages include Python for AI development, JavaScript/TypeScript and Node.js for web development, as well as C and C++. Additionally, he possesses strong working proficiency in both English and Persian.



Mahdi Eftekhari was born in Kerman, Iran, in 1978. He received his B.Sc. in Computer Engineering from the Department of Computer Science and Engineering at Shiraz University, Shiraz, Iran, in September 2001. He obtained his M.Sc. and Ph.D. degrees in Artificial Intelligence from the same department in 2004 and 2008, respectively. He has been a faculty member at Shahid Bahonar University of Kerman, Kerman, Iran, since 2008, where he is currently a Full Professor in the Department of Computer Engineering. His research interests include fuzzy methods and systems, machine learning, foundation models, and language models. He is the author or co-author of approximately 180 papers published in journals and conference proceedings.